# Bomb Lab Phase 3: jump table (switch)

Bomb Lab · gdb disassembly 분석

### Phase 3

다음은 phase_3의 disassembly와 gdb로 읽은 jump table이다. 폭탄을 해제하는 입력 2개를 찾아라.

```
callq __isoc99_sscanf          # 정수 2개 (개수 %eax로 반환)
cmp  $0x1, %eax
jg   <phase_3+39>             # 2개 이상 입력해야 함
cmpl $0x7, 0xc(%rsp)           # x1 ≤ 7 (unsigned)
ja   explode_bomb
mov  0xc(%rsp), %eax
jmpq *0x402540(,%rax,8)        # x1 값으로 jump table 분기
```

| index (x1) | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| 목적지 | +64 | +57 | +76 | +88 | +100 | +112 | +124 | +136 |

힌트 보기

switch는 jump table: jmp *base(,%rax,8). gdb x/8gx base로 테이블 엔트리(각 case 주소)를 읽고, case별 상수 연산을 추적해 마지막 cmp 조건 두 개를 모두 맞춘다.

정답 보기

### 풀이

① `x/8gx 0x402540`으로 테이블을 읽음: 각 엔트리가 case 몸통의 주소

② 각 case는 mov $0, %eax → add/sub 상수의 조합으로 %eax를 만듦. x1 = 4 (+100)면 `mov $0x0,%eax; add $0x3ac,%eax` → %eax = 0x3ac… 이어서 sub $0x3ac → %eax = 0

③ 마지막에 cmpl $0x5, 0xc(%rsp) (x1 ≤ 5)와 cmp 0x8(%rsp),%eax (x2 = %eax)를 통과해야 함
→ x1 = 4, x2 = 0

최종 답 **4 0**